

Lesson 2

ReactDOM, Components, JSX & Troubleshooting

Code the Dream — React

Today's Session

EXPLORE

- How ReactDOM connects React to the page
- What components are and how they're structured
- The component tree
- JSX syntax and its rules
- Reading error messages and React DevTools

APPLY

- Create three components in separate files
- Sort and render data with `.toSorted()`
- Assemble a Snack Rankings app

By the end: understand how React renders to the page, write valid JSX, and organize your code across multiple component files.

How React Gets on the Page

Three files work together every time your app loads:

index.html — the entry point Vite serves.
Contains one `<div id="root">` where React will render.

main.jsx — imports React and your App component, then mounts the app into `#root`.

```
// main.jsx
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

createRoot(document.getElementById('root'))
  .render(<App />);
```

`createRoot` finds the `#root` div and hands control to React. Everything rendered by your app lives inside that div.

StrictMode

You'll notice `<StrictMode>` wrapping `<App />` in the template's `main.jsx`.

- A **development-only** tool — it has no effect in production builds
- Mounts components **twice** to surface impure render bugs
- Flags use of deprecated React APIs

```
createRoot(document.getElementById('root')).render(  
  <StrictMode>  
    <App />  
  </StrictMode>,  
);
```

If you see effects running twice or double console logs, that's StrictMode working as intended — don't remove it.

What Is a Component?

A component is a **JavaScript function that returns JSX**.

```
function Greeting() {  
  return <h1>Hello, world!</h1>;  
}
```

Components can be written three ways — all are valid:

```
function Welcome() { return <p>Function declaration</p>; }
```

```
const Welcome = function() { return <p>Function expression</p>; };
```

```
const Welcome = () => <p>Arrow function (implicit return)</p>;
```

Component names must start with a **capital letter**. React uses this to tell components apart from plain HTML tags.

Component Rules & Best Practices

Rules (required):

- Name must be **capitalized**
- Must return a **single root element** (wrap siblings in `<>...</>`)
- Must be a **pure function** — same inputs always produce the same output
- Do not directly mutate props or state

Best practices:

- Use **PascalCase** for multi-word names — e.g. `TodoList`, `BackButton`
- One component per file; file name matches component name
- Export with `export default ComponentName` at the bottom

The Component Tree

Components nest inside each other to form a **tree**.

```
root (main.jsx)
└─ App
    ├── Header
    │   └─ ShoppingCart
    └─ ProductList
        ├── Product
        ├── Product
        └─ Product
```

- **Parent** components contain child components
- Data flows **down** the tree via props (more on this in Lesson 3)
- Each component is responsible for **one part** of the UI

Check for Understanding

Which of the following is **not** a valid React component?

```
function myComponent() { return <p>Hello</p>; }           // A
const MyComponent = () => <p>Hello</p>;                   // B
function MyComponent() { return <p>Hello</p>; }           // C
const MyComponent = function() { return <p>Hello</p>; };  // D
```

A) Option A B) Option B C) Option C D) Option D

JSX: JavaScript + HTML

JSX (JavaScript XML) lets you write HTML-like syntax inside JavaScript. Vite transforms it to plain JS before the browser sees it.

```
// What you write
function App() {
  return (
    <div className="app">
      <h1>CTD Swag</h1>
    </div>
  );
}
```

JSX is easier to read and write — but it's just JavaScript under the hood.

JSX: What Vite Compiles It To

Under the hood, every JSX element becomes a `React.createElement` call.

```
// What the browser actually runs
React.createElement(
  'div',
  { className: 'app' },
  React.createElement('h1', null, 'CTD Swag')
);
```

You never need to write `React.createElement` by hand — that's the whole point of JSX. This is just to show what Vite is doing for you.

JSX Rules — Structure

1. Return a single root element. Wrap siblings in a Fragment `<>...</>`.

```
// Invalid – two siblings with no wrapper  
return <h1>Title</h1><p>Text</p>;
```

```
// Valid – wrapped in a Fragment  
return (  
  <>  
    <h1>Title</h1>  
    <p>Text</p>  
  </>  
)
```

2. All tags must be closed — either with a closing tag or self-closing.

```
    {/* self-closing */}  
<input type="text" />    {/* self-closing */}
```

JSX Rules — Attributes & Expressions

3. Attribute names are camelCase (with two exceptions).

HTML	JSX
class	className
for	htmlFor
onclick	onClick
aria-label, data-id	unchanged

4. JavaScript expressions go inside {}.

```
const name = "Alex";
return <p>Hello, {name}!</p>;

// Objects need double braces {{ }}
return <div style={{ color: 'navy', fontSize: 18 }}>Styled</div>;
```

Conditional Rendering

JSX only supports **expressions**, not statements — so we use the ternary operator or `&&` to conditionally render content.

```
{/* Ternary: show one thing or another */}  
{isLoggedIn ? <p>Welcome back!</p> : <button>Log In</button>}
```

```
{/* && : show something or nothing */}  
{hasError && <p className="error">Something went wrong.</p>}
```

```
{/* Mapping a list */}  
{items.map((item) => (  
  <li key={item.id}>{item.name}</li>  
))}
```

Check for Understanding

Which line contains a JSX error?

```
function Card() {  
  return (  
    <div class="card">  
        
      <p>Hello!</p>  
    </div>  
  );  
}
```

- A) Line 3 — `class` should be `className`
- B) Line 4 — `<img>` must be self-closing: ``
- C) Both A and B
- D) There are no errors

Reading Error Messages

When something breaks, React and Vite give you clues — learn to read them.

In the browser: A red overlay appears with the error type and a short description.

In the terminal: The same error is printed, often with a ^ caret pointing to the exact problem character.

Common errors to recognize:

Error	Likely cause
Unexpected token	Missing closing tag or }
is not defined	Forgot to import a component
Warning: Each child should have a unique key	.map() missing key prop

React Developer Tools

Install the **React DevTools** browser extension to inspect your component tree and state in real time.

- [Chrome](#)
- [Firefox](#)
- [Edge](#)

Once installed, open browser DevTools → look for the **Components** and **Profiler** tabs.

The **Components** tab lets you click any component in the tree to see its props and state — invaluable for debugging.

Now: Apply

Let's build a multi-component app from scratch.

This Week's Exercise — Snack Rankings App

You'll build a **Snack Rankings** app organized across multiple component files.

In `src/exercises/lesson-02/` **create:**

- `SnackHeader.jsx` — a heading for the app
- `SnackList.jsx` — a sorted list of your favorite snacks
- `SnackFooter.jsx` — a fun footer message

Then: assemble all three in `studentWork.jsx`.

One component per file. File name matches the component name. This is the pattern you'll use all course.

Step 1 — SnackHeader & SnackFooter

These are simple — one job each. Create both files in `src/exercises/lesson-02/`.

SnackHeader.jsx

```
function SnackHeader() {  
  return <h1>My Snack Rankings</h1>;  
}  
export default SnackHeader;
```

SnackFooter.jsx

```
function SnackFooter() {  
  return <p>No debates accepted.</p>;  
}  
export default SnackFooter;
```

Save both — no errors should appear in the browser.

Step 2 — SnackList

Create an array of snack objects, sort it, and render the result.

```
// SnackList.jsx
function SnackList() {
  const snacks = [
    { name: "Pretzels", rank: 3 },
    { name: "Chips", rank: 1 },
    { name: "Popcorn", rank: 2 },
  ];

  const sorted = snacks.toSorted((a, b) => a.rank - b.rank);

  return (
    <ol>
      {sorted.map((snack) => (
        <li key={snack.name}>{snack.name}</li>
      ))}
    </ol>
  );
}

export default SnackList;
```

Step 3 — Assemble in studentWork.jsx

Import all three components and render them together.

```
// studentWork.jsx
import SnackHeader from './SnackHeader';
import SnackList from './SnackList';
import SnackFooter from './SnackFooter';

export default function StudentWork() {
  return (
    <div>
      <SnackHeader />
      <SnackList />
      <SnackFooter />
    </div>
  );
}
```

Save and check the browser — your ranked snack list should appear under Lesson 2.

Extension — Push Your Components Further

If you finish early, try one or more of these:

- Add a **second array** of drinks and a `DrinkList` component — import and render it alongside `SnackList`
- Add a **rank badge** to each list item: `<span>#{snack.rank}</span>`
- Use a **ternary** to show a different message in `SnackFooter` based on how many snacks are in the list (e.g. fewer than 3 vs. 3 or more)

Wrap-Up

This week you learned:

- `createRoot` in `main.jsx` mounts your app into `index.html`'s `#root` div
- Components are JavaScript functions that return JSX — one per file, capitalized
- JSX has strict rules: single root, closed tags, camelCase attributes, `{}` for expressions
- Error messages and React DevTools are your debugging toolkit

Before next session: Complete the weekly assignment — create `TodoList` and `TodoForm` components and integrate them into your app. Submit your PR link in CTD Learns. Post questions in **`#[class-name]-discussion`** on Slack.